01076106 Object Oriented Programming 2/2568
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ข้อกำหนด Mini Project

1. เขียนด้วยภาษา Python โดยใช้หลักการ OOP โดยมีเกณฑ์ของ OOP ดังนี้
    - ต้องมี Abstract Class อย่างน้อย 1 คลาส โดยต้องใช้อย่างเหมาะสมด้วย
        - โดยให้มี Inheritance จำนวน 2 ลำดับชั้นขึ้นไป
        - มีการใช้ Polymorphism จำนวน 1 จุด
    - ต้องมี Class มากกว่า 10 คลาสขึ้นไป โดยไม่นับ Class ที่ Inherit ที่ไม่มี Logic ข้างในมากพอ เช่น Inherit เพื่อเป็นช่องทางการชำระเงิน (กรณีนี้จะนับเฉพาะ Parent Class)
        - ต้องมี class ที่มีลักษณะเก็บข้อมูลการทำรายการ (Transaction) Class เช่น การจอง การซื้อสินค้า อย่างน้อย 2 คลาส
        - ต้องมี Controller Class เพียง 1 Class
        - ต้องมี Entity Class (คลาสที่เก็บข้อมูลพื้นฐาน) อย่างน้อย 3 คลาส โดยแต่ละคลาสจะต้องมี ID และต้องมีจำนวนจำกัด คือ สามารถระบุจำนวนที่ใช้ไป และจำนวนที่เหลือได้
        - ต้องมี Class ที่มีการติดตามสถานะ อย่างน้อย 3 สถานะ จำนวน 1 Class ขึ้นไป
        - ต้องไม่มี setter/getter ที่ไม่มีการเรียกใช้งาน
        - ทุก attribute ต้องเป็น private ทั้งหมด
    - มีกฎธุรกิจ
        - จำกัดจำนวน หรือ จำกัดเวลา จำนวน 2 กฎขึ้นไป
        - มีองค์ประกอบค่าใช้จ่ายอย่างน้อย 4 รายการ
        - สิทธิ์/คูปอง/ส่วนลด อย่างน้อย 1 ประเภท
    - มี Error Handling เพื่อให้โปรแกรมทำงานได้เสถียร
        - มีการตรวจสอบ input (input validation) ทุกแห่ง
        - มีการใช้ try-except อย่างน้อย 2 จุด
    - Code ต้องอยู่ในไฟล์ 2 ไฟล์ขึ้นไป
2. รายละเอียดของ diagram
    - Use Case Diagram
        - มี Actor ที่เป็นคนอย่างน้อย 2 Actor และ/หรือ Actor ภายนอก 1 Actor โดยให้รวมได้ 3 Actor ขึ้นไป โดยแต่ละ Actor ควรมี 2 Use Case เป็นอย่างน้อย โดย Use Case ควรมีความซับซ้อนในตัวเองระดับหนึ่ง
        - มี Use Case จำนวน 12 Use Case ขึ้นไป โดยต้องเขียนในกรอบสี่เหลี่ยม เขียนชื่อระบบ Actor ต้องอยู่ภายนอกกรอบ
        - มีการใช้ Include / Extend Relationship ตามความเหมาะสม โดยกรณีที่เป็นการทำงานต่อเนื่องกัน ควรรวมเป็น Use Case เดียวกัน เพื่อลดจำนวน User Case และให้เหมาะกับลักษณะโปรแกรม

    - มี Generalization จำนวน 1 แห่ง
    - ให้ระบายสี Use Case โดยสีเหลืองอ่อนคือ Use Case ง่าย ไม่ต้องนำไปเขียน Sequence Diagram และสีฟ้าอ่อน คือ Use Case ที่มีความซับซ้อนกว่า เกี่ยวข้องกับธุรกรรม เช่น การจอง การซื้อของ ซึ่งเป็น Use Case ที่จะนำไปเขียน Sequence Diagram ซึ่งจำนวนของ Use Case สีฟ้าอ่อน ประมาณ 2 x จำนวนคนในกลุ่ม โดยหาก Use Case ที่เกี่ยวข้องกับธุรกรรมไม่เพียงพอ ให้พิจารณาเพิ่ม Use Case ธุรกรรม แต่หากไม่ได้ก็อนุโลมให้ใช้ Use Case สีเหลือง เท่าที่จำเป็นเท่านั้น
    - Diagram ต้องอยู่ใน A4 1 แผ่น และ แยกคำอธิบายออกไปต่างหาก
- Class Diagram
    - ต้องแสดง Class ของระบบที่ครบถ้วน มี Attribute และ Method (ไม่รวม getter/setter)
    - ต้องแสดงความสัมพันธ์ครบถ้วน โดย Class ที่มี attribute ใดที่ไม่ใช่ของตนเอง จะต้องมีเส้นความสัมพันธ์ไปยัง Class อื่นให้ครบ และ แสดงความสัมพันธ์ที่เหมาะสม ความสัมพันธ์ทุกเส้นจะต้องมี Multiplicity
    - ทุก Class ต้องระบุ Attribute, Method และความสัมพันธ์ให้ครบ และ จะต้องตรงตาม Code โดยข้อมูลที่เกี่ยวกับคลาสอื่นต้องเก็บเป็น Instance ห้ามเก็บข้อมูลเป็น dictionary หรือ list ซ้อน list หากพิจารณาแล้วจำเป็นต้องใช้ ให้สร้างเป็น class ที่ดูแลความสัมพันธ์ของ list ที่ซ้อนกันแทน
    - กรณีที่มีค่าคงที่ ให้พิจารณาใช้เป็น Enum Class
- Sequence Diagram
    - ทุก Use Case ที่เป็นสีฟ้าอ่อน จะต้องเขียน Sequence Diagram ทั้งหมด โดยตัวอักษรใน Sequence Diagram เมื่อลงในรายงานแล้วจะต้องอ่านออก
    - ในแต่ละ Sequence Diagram จะต้องอยู่ใน 1 แผ่น A4 แนวตั้งหรือแนวนอนก็ได้ และต้องอ่านออก โดยต้องมีคลาสที่เป็นธุรกรรม เป็นส่วนหนึ่งของ Diagram โดยต้องมี Class ใน Diagram อย่างน้อย 4 คลาส และมี Step การทำงาน (ไปกลับ) อย่างน้อย 4
    - ควรมี Sequence ที่มีรายละเอียด fragment ตามลำดับจริง ไม่น้อยกว่า 2
    - ทุกลำดับการทำงานต้องแสดง method ที่เรียก (ถ้าให้ดีรวม argument ด้วย แต่ถ้ายาวสามารถละได้) โดย method จะต้องตรงกับใน Class Diagram และ Code

3. การทดสอบการทำงาน
    - ในแต่ละ Use Case จะต้องทดสอบการทำงาน โดยให้สมมติ Scenario หรือลำดับการทำงานให้ครบทุกขั้นการทำงาน โดยผ่าน AI Chat และจะต้องแสดงชื่อของ use case และ method ของแต่ละคลาสที่เรียกใช้ตามลำดับ เพื่ออธิบายให้ตรงกับ sequence diagram ของ use case นั้นๆ เช่น ถ้าในระบบ Co-Working Space จะมีลำดับการทำงานดังนี้
        - **Phase A: Discovery & Booking (การค้นหาและจอง)**
            1. **search_space**: ค้นหาห้องหรือโต๊ะที่ว่างตามเงื่อนไข
            2. **reserve_space**: ทำการจองล่วงหน้าสำหรับสมาชิก
            3. **view_booking**: ตรวจสอบรายละเอียดการจองที่เพิ่งทำไป

- **Phase B: On-site Operations (การเข้าใช้งานหน้างาน)**
    1. **book_space_walkin**: (ในกรณีสวมบทบาทเจ้าหน้าที่) จองให้ลูกค้าที่เดินเข้ามาโดยไม่ได้จองล่วงหน้า
    2. **check_in**: เปลี่ยนสถานะจาก "จองแล้ว" เป็น "กำลังใช้งาน"
- **Phase C: Completion & Billing (การคืนพื้นที่และชำระเงิน)**
    1. **checkout**: แจ้งออกจากพื้นที่เพื่อให้ระบบคำนวณเวลา/ค่าใช้จ่าย
    2. **payment**: ดำเนินการชำระเงินตามยอดที่สรุป
    3. **cancel_space**: (กรณีทดสอบข้อยกเว้น) ลองยกเลิกรายการจองที่ยังไม่เกิดขึ้น

- จากนั้นให้สร้าง Prompt สำหรับทดสอบในลำดับการทำงานข้างต้น เช่น
    - **ค้นหาและจอง**"ช่วยหาห้องประชุมขนาด 4 คนที่ว่างช่วงบ่าย 2 วันนี้ให้หน่อย ถ้ามีให้จองในชื่อ 'อาจารย์สมชาย' เลยครับ"
    - **ตรวจสอบรายการ**"ขอดูรายละเอียดการจองทั้งหมดของผมหน่อยว่ามีเลขที่การจอง (Booking ID) อะไรบ้าง"
    - **การ Walk-in**"ตอนนี้มีลูกค้า walk-in มาหน้าเคาน์เตอร์ ชื่อคุณบี อยากได้โต๊ะเดี่ยว 1 ที่ ช่วยทำเรื่องจองให้หน่อยในฐานะเจ้าหน้าที่"
    - **Check-in**"ผมมาถึง coworking space แล้ว รหัสการจอง BK123 ช่วยเช็คอินให้ผมหน่อยครับ"
    - **Check-out & Pay**"ผมเลิกใช้งานแล้วครับ ช่วยเช็คเอาต์รหัส BK123 และดำเนินการสรุปยอดชำระเงินให้ด้วย"
    - **ยกเลิกการจอง**"ขอยกเลิกการจองรหัส BK456 ที่ผมจองไว้พรุ่งนี้หน่อยครับ"
- นักศึกษาสามารถสร้าง Prompt chat ที่ซับซ้อน ที่อาจเรียกการทำงานของหลายๆ service มาทำงานร่วมกันได้ ซึ่งจะได้คะแนนเป็นพิเศษ แต่ต้องแสดงลำดับการทำงาน ได้ โดยสามารถแสดง method ที่ต้องถูกเรียกตามลำดับใน mcp tool ที่เรียกใช้

4. การส่งงาน

- รายงาน 1 เล่ม โดยเป็นไฟล์ PDF ประกอบด้วย
    - บทที่ 1 บทนำ เล่าถึงระบบโดยรวม อาจนำไฟล์ Spec มาสรุปใส่ สามารถใช้ AI สรุปได้
    - บทที่ 2 Use Case Diagram มีภาพของ Use Case diagram และ Use Case Description ตามตัวอย่าง

Use Case name	
Actor	
Descriptions	

    - บทที่ 3 Class Diagram
    - บทที่ 4 รายละเอียดการทำงาน ให้แสดงการทำงานในแต่ละ Use Case Sequence Diagram ที่เรียงตามเรื่องราว ตามข้อ 3 ประกอบด้วย
        1. ชื่อ Use Case และอธิบายหน้าที่ของ Use Case นี้ พร้อมทั้งพารามิเตอร์ที่ส่ง และรายละเอียดของ API/service ที่เกิดจาก Use Case นั้น
        2. Sequence Diagram พร้อมคำอธิบายโดยย่อ
        3. Code ใน Class ส่วนที่ตรงกับ Sequence Diagram โดยการ Capture จะต้องไม่ทำให้ Code มีขนาดเล็กจนอ่านยาก
        4. Code ในส่วนของ MCP Service ที่เกี่ยวข้อง โดยการ Capture จะต้องไม่ทำให้ Code มีขนาดเล็กจนอ่านยาก
        5. ถ้าทดสอบเป็นแบบ FastAPI ก่อน ให้ Capture หน้า Swagger มาด้วย หรือใช้ mcp-cli
        6. Prompt ที่ใช้ทดสอบการทำงาน และ ผลลัพธ์ของการทำงาน ให้ Capture ให้อ่านได้
- Use Case Diagram จำนวน 1 หน้า 1 ไฟล์ โดยเป็นไฟล์ PDF
- Class Diagram จำนวน 1 หน้า 1 ไฟล์ โดยเป็นไฟล์ PDF
- Sequence Diagram จำนวน 1 Diagram ต่อหน้าต่อไฟล์ (ตั้งชื่อตาม use case) โดยรวมเป็นไฟล์ทั้งหมดเป็น zip จำนวน 1 ไฟล์
- ไฟล์ Code โดยให้ส่งดังนี้
    - ไฟล์ code python ทุกไฟล์ในโครงงาน โดยให้ตั้งชื่อไฟล์หลักว่า project.py เก็บใส่ folder ชื่อ source
    - หากมีการใช้ Library อื่นๆ นอกจาก Lib มาตรฐาน และ FastAPI หรือ MCP ให้สร้างเป็นไฟล์ requirement.txt ด้วย
    - ไฟล์ config ของ mcp-cli และของ claude เพื่อการทดสอบ
    - ไฟล์ prompt.txt หรือ prompt.md สำหรับเก็บ prompt ที่ในการทดสอบตามลำดับ
    - Zip ทุกไฟล์รวมกัน
- แสดงการทำตามเงื่อนไขในข้อ 1 ในห้องเรียน ในช่วงเวลาสาธิต

- การตั้งชื่อไฟล์ group_name_0123_0124_0125_0126_type.pdf โดย group_name เป็นชื่อกลุ่มแบบย่อ (เพราะบางกลุ่มยาวมาก) ตามด้วยรหัสนักศึกษา 4 หลักของสมาชิก และ type คือ ประเภทไฟล์ เช่น usecase, class, code, report, sequence

5. ส่วนของการทำงานกลุ่ม
    - ให้ capture หน้า Github ของโครงงาน ในหน้าแรกและหน้า insight โดยทุกคนควรมีส่วนร่วมไม่น้อยกว่า 20 % โดยให้ทำแบบสำรวจ peer evaluation ที่อาจารย์กำหนด
6. การตรวจ
    - จะตรวจในวันที่ 10-11 มี.ค. ประกอบด้วยการ นำเสนอการทำงาน ผ่าน Flow ที่กำหนดขึ้น และ ตอบคำถามอาจารย์ในการทำงานของ Code

Rubric สำหรับให้คะแนน

1. Use Case Diagram (10 คะแนน)

เกณฑ์	**ระดับ 4 (10)**	**ระดับ 3 (8)**	**ระดับ 2 (6)**	**ระดับ 1 (0–4)**
ความครบถ้วนตามเงื่อนไข	Actor ≥ 3, Use Case ≥ 12, มี include/extend เหมาะสม, มี generalization 1 จุด, ระบายสีถูกต้อง	ขาดเงื่อนไขเล็กน้อย 1 จุด (เช่น use case 11 หรือ actor 2)	ขาดหลายจุด (เช่น ไม่มี generalization หรือสีไม่ตรงแนวคิด)	ไม่ตรงเงื่อนไขหลัก หรือ diagram ใช้งานไม่ได้
คุณภาพเชิงระบบ	Use Case มีความซับซ้อนเหมาะสม include/extend ถูกต้อง ไม่ปั่นจำนวน	มีจุด include/extend ไม่เหมาะสมเล็กน้อย	Use Case หลายอันซ้ำซ้อน/ไม่สื่อความหมาย	ไม่สะท้อนระบบจริง อ่านแล้วไม่เข้าใจ
ความชัดเจนและการนำเสนอ	อยู่ใน A4 1 หน้า อ่านชัดเจน ชื่อระบบ/actor ถูกตำแหน่ง	อ่านได้แต่แน่น/เล็กบางจุด	อ่านยาก มีความรก	อ่านไม่ได้/ผิดรูปแบบ

2. Class Diagram (25 คะแนน)

เกณฑ์	**ระดับ 4 (25)**	**ระดับ 3 (20)**	**ระดับ 2 (15)**	**ระดับ 1 (0–10)**
ความครบถ้วนของคลาส	แสดงคลาสครบตามระบบ และตรงตาม code	ขาดคลาสบางส่วนแต่ยังเห็นภาพรวม	ขาดคลาสสำคัญ 1 จุดขึ้นไป	Diagram ไม่ครอบคลุมระบบ
Attribute/Method	มี attribute/method ครบ (ไม่รวม getter/setter) และสื่อความหมาย	มีบ้างที่ตกหล่น/ตั้งชื่อไม่ดี	method/attribute หลายคลาสไม่ชัดเจน	แทบไม่มี method/attribute หรือผิดแนว UML
ความสัมพันธ์และ Multiplicity	ความสัมพันธ์ครบทุกเส้น + multiplicity ทุกเส้นถูกต้อง	ความสัมพันธ์ครบ แต่ขาด multiplicity บางเส้น	ความสัมพันธ์หลายเส้นผิด/หาย	ไม่มีความสัมพันธ์ที่ใช้จริง
สอดคล้องกับ OOP Design	เห็น abstraction, inheritance, polymorphism, enum/constant ถูกใช้เหมาะสม	มี OOP design แต่ยังไม่คมบางจุด	OOP design ไม่ชัด หรือยัด inheritance	ออกแบบไม่เป็น OOP
ความชัดเจนและการนำเสนอ	อยู่ใน A4/A3 1 หน้า อ่านชัดเจน	อ่านได้แต่แน่น/เล็กบางจุด	อ่านยาก มีความรก	อ่านไม่ได้/ผิดรูปแบบ

3. Sequence Diagram (25 คะแนน)

เกณฑ์	ระดับ 4 (25)	ระดับ 3 (20)	ระดับ 2 (15)	ระดับ 1 (0–10)
ครบตาม Use Case สีฟ้า	ทำครบทุก Use Case สีฟ้า (ตามที่กำหนด)	ขาด 1 use case	ขาดหลาย use case 2 use case ขึ้นไป	ขาดมากกว่า 3 use case หรือทำไม่ตรง use case
โครงสร้าง diagram	แต่ละหน้า A4 อ่านออก ชัดเจน มี class ≥ 4 และ step ≥ 4 (ไปกลับ)	ครบเงื่อนไขแต่แน่น/อ่านยากเล็กน้อย	มี class < 4 และ step < 4	Diagram ใช้งานไม่ได้/ไม่อ่านออก
ความถูกต้องของ message/method	message แสดง method ตรงกับ class diagram + code	มีบาง method ไม่ตรงกัน ไม่เกิน 2 จุด -	class diagram หรือ code ไม่ตรงกัน ไม่เกิน 4 จุด	ไม่ตรงกันเกือบทั้งหมด
ความสมบูรณ์ของ diagram	มีโครงสร้างครบสมบูรณ์ ทั้ง Actor, Activation Bar, Loop, If, Create ตามที่ควรจะมีใน Diagram	โครงสร้างส่วนใหญ่สมบูรณ์ มีตกหล่น ไม่เกิน 2 จุด	โครงสร้างไม่ค่อยสมบูรณ์ ตกหล่นไม่เกิน 4 จุด	ไม่เป็นโครงสร้างเลย
ความเป็นธุรกรรม	มี transaction class อยู่ใน diagram และ flow ตามการใช้งานจริง	มี transaction class แต่ลำดับ method ที่เรียกใช้ไม่ครบ ขาดไม่เกิน 2 method	transaction class อยู่โดยไม่จำเป็นต้องมี	ไม่มี transaction หรือ flow ไม่สมเหตุผล

4. Coding (25 คะแนน)

เกณฑ์	ระดับ 4 (25)	ระดับ 3 (20)	ระดับ 2 (15)	ระดับ 1 (0–10)
OOP ตามเงื่อนไข	มี abstract class, inheritance ≥2 ชั้น, polymorphism 1 จุด ใช้เหมาะสม	ครบแต่ไม่สมเหตุสมผลตามหลักการ เช่น ข้อมูลเหมือนกันแต่แยกเพื่อให้มีครบ	มีแต่ไม่ถึง 2 ชั้นหรือไม่มี polymorphism	ไม่ครบเงื่อนไขหลัก
จำนวนและชนิดของคลาส	>10 คลาสจริง มี transaction ≥2, controller 1, entity ≥3 (มี ID + ไม่จำกัดจำนวน)	ขาดบางอย่าง 1 จุด	ขาด 2 จุดขึ้นไป	ขาดมากกว่า 4 จุด
Business Rules	มีกฎจำกัด ≥2 + ค่าใช้จ่าย ≥4 + ส่วนลด/คูปอง ≥1 ใช้จริง	มีครบแต่ใช้งานไม่ครบทุกส่วน	มีบางส่วนแต่ไม่ครบจำนวน	ไม่มีหรือไม่ทำงานจริง
Error Handling	validation ทุก input + try-except ≥2 จุด และครอบคลุมเคสจริง	validation ตกหล่นเล็กน้อย	validation ขาดหลายจุด	ไม่มี validation / crash ง่าย

คุณภาพโค้ด	โค้ดอ่านง่าย แยกไฟล์ ≥2, private attribute, ไม่มีกลุ่ม getter/setter ที่ไม่ใช้	โค้ดพออ่านได้ มีความซ้ำบางจุด	โค้ดเริ่มรก/ซ้ำเยอะ/โครงสร้างไม่ดี	โค้ดไม่เป็นระบบ/แก้ยาก
ความสมบูรณ์ของการทำงาน	ทำงานได้ครบตาม Requirement ที่แจ้งโดยสมบูรณ์	มีการทำงานได้ไม่ครบถ้วน ขาดไม่เกิน 2	มีหลายจุดที่ทำงานได้ไม่ครบถ้วน ขาดไม่เกิน 4	ทำงานไม่ค่อยได้

5. รายงาน (15 คะแนน)

เกณฑ์	**ระดับ 4 (15)**	**ระดับ 3 (12)**	**ระดับ 2 (9)**	**ระดับ 1 (0–6)**
โครงสร้างรายงาน	ครบทุกส่วน: บทนำ, use case+desc, class, sequence ตาม flow, prompt+ผลลัพธ์	ขาดบางส่วนไม่เกิน 2	ขาดหลายส่วน ไม่เกิน 4	รายงานไม่ครบ/ไม่เป็นรูปเล่ม
ความชัดเจนของภาพ/การ capture	ภาพอ่านออกทุกหน้า code คมชัด ขนาดตัวอักษรสามารถขยายแล้วยังอ่านได้ชัดเจน	อ่านไม่ชัดเจน ไม่เกิน 2	อ่านไม่ชัดเจน ไม่เกิน 4	อ่านไม่ได้
การทดสอบตามข้อ 3	มี scenario ครบทุก use case และ prompt + output ครบ	ขาด scenario ไป 1 use case	ขาด scenario มากกว่า 2 use case	ไม่แสดงการทดสอบจริง
ความสอดคล้องของเอกสาร	Diagram ↔ Code ↔ รายงาน ตรงกัน	มี mismatch ไม่เกิน 2 จุด	mismatch ไม่เกิน 4 จุด	ไม่ตรงกันจนตรวจไม่ได้